

QMS Raspberry Pi Deployment Flow Guide

GitHub-based remote software deployment for the MQTT Queue Manager

Repository: sonvisageabuja1/QMS_Pi_code

Main application: /home/sonvisage/QMS_Pi_code/mqtt_queue_manager.py

Runtime service: mqtt_queue.service

Update timer: qms-update.timer

Update interval: every 5 minutes

DEVELOPMENT DEVICE	GITHUB	DEPLOYMENT PIs
Mac / Windows / Dev Pi	main branch	Pi #1 / Pi #2 / Pi #3
Edit, test, commit, push	Approved code	Check every 5 minutes
		Fetch + fast-forward
		Restart MQTT service

1. Current QMS Layout

```
/home/sonvisage/  
|-- mqtt-env/  
`-- QMS_Pi_code/  
    |-- mqtt_queue_manager.py  
    |-- rfid_mappings.txt  
    |-- esp32_mqtt_listener.py  
    |-- mqtt_logger.py  
    |-- queues.json  
    `-- .gitignore
```

queues.json is live runtime data and is ignored by Git. Historical versioned Python files are also ignored.

2. GitHub and SSH

```
git@github.com:sonvisageabuja1/QMS_Pi_code.git  
Branch: main
```

Each development device should use its own GitHub authentication. Never copy a private SSH key from one computer or Pi to another.

3. MQTT Queue Manager Service

```
[Unit]  
Description=MQTT Queue Manager Service  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
Type=simple  
User=sonvisage  
WorkingDirectory=/home/sonvisage/QMS_Pi_code  
ExecStart=/home/sonvisage/mqtt-env/bin/python /home/sonvisage/QMS_Pi_code/mqtt_queue_manager.py  
Restart=always  
RestartSec=5  
Environment=PYTHONUNBUFFERED=1  
  
[Install]  
WantedBy=multi-user.target
```

4. Automatic GitHub Update

qms-update.timer runs every five minutes. If origin/main has not changed, nothing is restarted. If a new commit exists, the Pi fast-forwards to it and restarts mqtt_queue.service.

```
sudo /usr/local/bin/qms-update.sh
systemctl status qms-update.timer --no-pager
journalctl -u qms-update.service -n 50 --no-pager
```

5. Normal Release from the Development Pi

```
cd /home/sonvisage/QMS_Pi_code
nano mqtt_queue_manager.py

sudo systemctl restart mqtt_queue.service
systemctl status mqtt_queue.service --no-pager

git status
git add mqtt_queue_manager.py
git commit -m "Describe the QMS update"
git push origin main
```

6. Uploading Code Remotely from a Different Device

You can publish QMS updates from a MacBook, Windows PC, or another development Raspberry Pi. Deployment Pis only need internet access to GitHub; you do not need to log into every Pi to deliver the update.

A. First-time setup on the new development device

```
ssh-keygen -t ed25519 -C "QMS-Development-Device"
cat ~/.ssh/id_ed25519.pub

# Add the public key to your GitHub account, then:
ssh -T git@github.com

git clone git@github.com:sonvisageabuja1/QMS_Pi_code.git
cd QMS_Pi_code
```

Important: Do not copy the Raspberry Pi private key to another device. Generate a separate SSH key on each development computer.

B. Make and publish an update

```
cd QMS_Pi_code

# Edit with VS Code, nano, or another editor
git status
git diff

git add mqtt_queue_manager.py
git commit -m "Update QMS remotely"
git push origin main
```

After the push reaches GitHub, configured deployment Pis detect the new commit during their next five-minute check, download it, and restart the MQTT service.

7. Recommended Access Model

Device type	GitHub permission	Purpose
MacBook / Windows PC / Development Pi	Read + Write	Develop, test, commit and push
Deployment Raspberry Pi	Read only	Pull approved releases automatically

For field/deployment units, read-only deploy keys are recommended. Development devices can retain push access.

8. Safer Release Workflow

```
DEVELOPMENT DEVICE
|
v
Edit code
|
v
Test locally
|
v
git commit
|
v
git push origin main
|
v
GitHub main
|
v
Deployment Pis check every 5 minutes
|
v
New commit -> update -> restart service
```

Because every approved push to main is automatically deployed, test changes before pushing. For a larger installation, a separate development/testing branch can be introduced before merging approved releases into main.

9. RFID Mapping Updates

```
git add rfid_mappings.txt
git commit -m "Update RFID mappings"
git push origin main
```

10. Diagnostics

```
systemctl status mqtt_queue.service --no-pager
journalctl -u mqtt_queue.service -n 50 --no-pager
systemctl status qms-update.timer --no-pager
journalctl -u qms-update.service -n 50 --no-pager
sudo /usr/local/bin/qms-update.sh
git -C /home/sonvisage/QMS_Pi_code log -1 --oneline
git -C /home/sonvisage/QMS_Pi_code status
```

11. Safety Behavior

If GitHub or the internet is unavailable, the existing QMS remains running. If no new commit exists, the service is not restarted. Live queues.json data is not managed by Git, so normal software deployments do not overwrite the active queue state.